

**UNIVERZITET U ZENICI
POLITEHNIČKI FAKULTET**

NUMERIČKA APROKSIMACIJA
SEMINARSKI RAD

STUDENTI

Safet Imamović
Emrah Jamaković
Islam Habibović

PREDMETNI NASTAVNIK

r. prof. dr. Aleksandar Karač

Zenica 2025/2026.

Abstrakt

U ovom radu detaljno je opisan proces razvoja i matematička pozadina softverskog rješenja za numeričku aproksimaciju funkcija. Fokus rada je na implementaciji *Metode najmanjih kvadrata* (Least Squares Method) unutar modernog web okruženja. Aplikacija omogućava korisnicima unos eksperimentalnih podataka te pronalazak optimalne funkcije (linearne, kvadratne, polinomske, eksponencijalne ili stepene) koja najbolje opisuje trend podataka. Posebna pažnja posvećena je teorijskom izvođenju normalnih jednačina, analizi greške putem koeficijenta determinacije (R^2), te softverskoj arhitekturi baziranoj na Next.js i TypeScript tehnologijama. Rad sadrži i praktične primjere primjene razvijenog alata na realne inženjerske probleme.

Ključne riječi: Numerička aproksimacija, Metoda najmanjih kvadrata, Regresija, Next.js, TypeScript.

Sadržaj

1	UVOD	4
2	TEORIJSKI DIO	5
2.1	Definicija problema aproksimacije	5
2.2	Linearna regresija	5
2.3	Polinomska aproksimacija	6
2.4	Aproksimacija nelinearnim funkcijama (Linearizacija)	6
2.4.1	Eksponencijalna funkcija	7
2.4.2	Stepena funkcija	7
2.5	Kvalitet aproksimacije	7
3	APLIKACIJA	8
3.1	Tehnološki okvir (Tech Stack)	8
3.2	Arhitektura podataka	8
3.3	Implementacija numeričkih algoritama	9
3.3.1	Rješavanje linearnih sistema	9
3.3.2	Algoritam linearne aproksimacije	10
3.4	Validacija ulaza i numerička stabilnost	11
3.5	Korisničko sučelje (UI)	12
4	PRIMJERI PRIMJENE	13
4.1	Primjer 1: Hookeov zakon (Fizika)	13
4.2	Primjer 2: Kondenzator (Elektrotehnika)	15
4.3	Primjer 3: Karakteristika pumpe (Mašinstvo)	18

4.4	Primjer 4: Balistička putanja projektila (Fizika)	21
4.5	Primjer 5: Otpor fluida u zavisnosti od brzine (Mehanika fluida)	23
5	ZAKLJUČCI	26

1 UVOD

U inženjerskoj i naučnoj praksi rijetko se susrećemo sa analitički zadanim funkcijama koje savršeno opisuju neki prirodni fenomen. Umjesto toga, na raspolaganju imamo skup diskretnih podataka dobijenih mjerenjem:

$$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$$

Ovi podaci su često opterećeni mjernim šumom, greškama očitavanja ili nepravilnostima samog procesa. Zbog toga, pokušaj da se pronađe funkcija koja prolazi tačno kroz sve tačke (interpolacija) često daje nezadovoljavajuće rezultate, rezultirajući funkcijama koje naglo osciliraju između tačaka i nemaju fizikalnog smisla.

Rješenje ovog problema leži u **aproksimaciji**. Cilj aproksimacije nije pogoditi tačne vrijednosti u svakoj tački mjerenja, već pronaći "glatku" funkciju $f(x)$ koja prolazi *što je moguće bliže* zadatim tačkama, minimizirajući ukupnu grešku. Najrasprostranjeniji metod za rješavanje ovog problema je **Metoda najmanjih kvadrata** (MNK), koju je početkom 19. vijeka neovisno razvio Carl Friedrich Gauss.

Ovaj seminarski rad ima dva osnovna cilja:

1. Prikazati teorijsku osnovu metode najmanjih kvadrata, uključujući izvođenje formula za različite tipove funkcija.
2. Opisati implementaciju ove metode u obliku moderne, interaktivne web aplikacije koja studentima i inženjerima služi kao alat za brzo modeliranje podataka.

Kroz rad ćemo proći put od matematičkog formulisanja problema, preko rješavanja sistema linearnih jednačina u računarskom kodu, do vizualizacije rezultata u web pregledniku.

2 TEORIJSKI DIO

2.1 Definicija problema aproksimacije

Neka je dat skup od N parova podataka (x_i, y_i) , gdje $i = 1, \dots, N$. Tražimo funkciju $f(x; a_0, a_1, \dots, a_m)$ koja zavisi od parametara a_k , takvu da najbolje aproksimira zavisnost y od x . Definiramo **rezidual** (grešku) u svakoj tački kao razliku izmjerene i procijenjene vrijednosti:

$$r_i = y_i - f(x_i) \quad (2.1)$$

Osnovna ideja metode najmanjih kvadrata je minimizacija sume kvadrata ovih reziduala. Kvadrat se koristi kako bi se izbjeglo poništavanje pozitivnih i negativnih grešaka, te kako bi se veće greške "kaznile" više od malih. Funkcija kriterija S je:

$$S(a_0, \dots, a_m) = \sum_{i=1}^N (y_i - f(x_i))^2 \quad (2.2)$$

Optimalni parametri su oni za koje funkcija S ima globalni minimum. Da bismo našli minimum, parcijalne izvode funkcije S po svakom od parametara a_k izjednačavamo sa nulom:

$$\frac{\partial S}{\partial a_k} = 0, \quad k = 0, \dots, m \quad (2.3)$$

2.2 Linearna regresija

Najjednostavniji i najčešći oblik aproksimacije je linearna funkcija (pravac):

$$y = ax + b \quad (2.4)$$

Ovdje su parametri a (nagib) i b (odsječak). Funkcija greške je:

$$S(a, b) = \sum_{i=1}^N (y_i - (ax_i + b))^2 \quad (2.5)$$

Minimizacija po b :

$$\frac{\partial S}{\partial b} = \sum_{i=1}^N 2(y_i - ax_i - b)(-1) = 0 \quad (2.6)$$

$$= -2 \left(\sum y_i - a \sum x_i - \sum b \right) = 0 \quad (2.7)$$

$$\Rightarrow \sum y_i = a \sum x_i + Nb \quad (2.8)$$

Minimizacija po a :

$$\frac{\partial S}{\partial a} = \sum_{i=1}^N 2(y_i - ax_i - b)(-x_i) = 0 \quad (2.9)$$

$$= -2 \left(\sum x_i y_i - a \sum x_i^2 - b \sum x_i \right) = 0 \quad (2.10)$$

$$\Rightarrow \sum x_i y_i = a \sum x_i^2 + b \sum x_i \quad (2.11)$$

Dobili smo sistem od dvije linearne jednačine sa dvije nepoznate (a i b), poznat kao **sistem normalnih jednačina**:

$$\begin{bmatrix} N & \sum x_i \\ \sum x_i & \sum x_i^2 \end{bmatrix} \begin{bmatrix} b \\ a \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum x_i y_i \end{bmatrix} \quad (2.12)$$

2.3 Polinomska aproksimacija

U općem slučaju, kada podatke želimo aproksimirati polinomom stepena m :

$$P_m(x) = a_0 + a_1 x + a_2 x^2 + \dots + a_m x^m \quad (2.13)$$

Postupak je analogan. Parcijalnim deriviranjem sume kvadrata grešaka dobija se sistem od $m + 1$ linearnih jednačina. Matrica sistema A sastoji se od suma stepena x :

$$A = \begin{bmatrix} N & \sum x_i & \dots & \sum x_i^m \\ \sum x_i & \sum x_i^2 & \dots & \sum x_i^{m+1} \\ \vdots & \vdots & \ddots & \vdots \\ \sum x_i^m & \sum x_i^{m+1} & \dots & \sum x_i^{2m} \end{bmatrix} \quad (2.14)$$

Vektor slobodnih članova B je:

$$B = \begin{bmatrix} \sum y_i \\ \sum x_i y_i \\ \vdots \\ \sum x_i^m y_i \end{bmatrix} \quad (2.15)$$

Rješavanjem sistema $Aa = B$ (npr. Gaussovom metodom) dobijamo koeficijente polinoma.

2.4 Aproksimacija nelinearnim funkcijama (Linearizacija)

Mnogi prirodni procesi (hlađenje, radioaktivni raspad, rast populacije) prate eksponencijalne ili stepene zakone, a ne polinomske. Primjena metode najmanjih kvadrata direktno na nelinearne funkcije dovela bi do sistema nelinearnih jednačina koji se teško rješavaju. Zato koristimo postupak **linearizacije**.

2.4.1 Eksponencijalna funkcija

Model: $y = ae^{bx}$. Logaritmiranjem jednačine dobijamo:

$$\ln y = \ln a + bx \quad (2.16)$$

Uvodimo smjene varijabli:

$$Y = \ln y, \quad A = \ln a, \quad B = b$$

Dobili smo linearnu jednačinu: $Y = A + Bx$. Sada na transformisanim podacima $(x_i, \ln y_i)$ primjenjujemo standardnu linearnu regresiju kako bismo našli A i B . Konačni parametri su:

$$a = e^A, \quad b = B$$

2.4.2 Stepna funkcija

Model: $y = ax^b$. Logaritmiranjem:

$$\ln y = \ln a + b \ln x \quad (2.17)$$

Smjene:

$$Y = \ln y, \quad X = \ln x, \quad A = \ln a, \quad B = b$$

Linearna forma: $Y = A + BX$. Parametri se određuju regresijom nad parovima $(\ln x_i, \ln y_i)$.

2.5 Kvalitet aproksimacije

Kako bismo ocijenili koliko dobro odabrana funkcija opisuje podatke, koristimo **Koeficijent determinacije** (R^2):

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}} = 1 - \frac{\sum (y_i - f(x_i))^2}{\sum (y_i - \bar{y})^2} \quad (2.18)$$

Gdje je $\bar{y}$ srednja vrijednost y podataka.

- $R^2 = 1$: Idealno poklapanje.
- $R^2 = 0$: Model ne objašnjava varijabilnost podataka.

3 APLIKACIJA

U ovom poglavlju detaljno opisujemo tehničku realizaciju softverskog rješenja. Aplikacija "Approximation Picker" nije samo kalkulator; to je modularna platforma dizajnirana da bude proširiva i jednostavna za održavanje.

3.1 Tehnološki okvir (Tech Stack)

Prilikom izbora tehnologija vodili smo se kriterijima: brzina razvoja, performanse u pregledniku, i kvaliteta ekosistema.

- **Next.js 15 (React Framework):** Omogućava nam kreiranje brzih, "server-side rendered" (SSR) ili statičkih web stranica. Iako se naša aplikacija u potpunosti izvršava na klijentu (SPA - Single Page Application), Next.js pruža odličnu strukturu projekta i routing.
- **TypeScript:** Superskup JavaScripta koji dodaje statičke tipove. U kontekstu numeričke matematike, ovo je neprocjenjivo. Na primjer, TypeScript nam ne dozvoljava da slučajno saberemo broj i niz, ili da funkciji koja očekuje matricu proslijedimo vektor.
- **MathLive:** Web komponenta ('<math-field>') koja omogućava korisnicima unos formula na način kako se one pišu u matematici (WYSIWYG editor).
- **Cortex Compute Engine:** Moćan "engine" za parsiranje i evaluaciju simboličkih izraza. Koristimo ga da pretvorimo korisnikov unos (npr. $\sin(x) + 2$) u funkciju koja se može evaluirati za bilo koje x .
- **Plotly.js:** Industrijski standard za naučno iscrtavanje grafova na webu. Koristi WebGL za hardversku akceleraciju, što omogućava glatko iscrtavanje hiljada tačaka.

3.2 Arhitektura podataka

Srž naše aplikacije su tipovi podataka definisani u TypeScript-u. Oni osiguravaju konzistenciju kroz cijelu aplikaciju.

```
1 // Definicija jedne mjerne tačke
2 export interface DataPoint {
3   x: number
4   y: number
5   id: string // Jedinostveni identifikator za React listu
6 }
7
```

```

8 // Rezultat prorauna aproksimacije
9 export interface ApproximationResult {
10   type: CalculationType
11   coefficients: number[] // Izraunati parametri (a, b...)
12   polynomial: string // Latex reprezentacija za prikaz
13   rSquared: number // Kvalitet aproksimacije
14   points: DataPoint[] // Originalne tačke
15   fittedPoints: DataPoint[] // Tačke na krivoj (za crtanje)
16   steps: string[] // Koraci rjeavanja (step-by-step)
17 }

```

Listing 1: Glavni TypeScript interfejsi iz 'lib/types/index.ts'

3.3 Implementacija numeričkih algoritama

Svi algoritmi su implementirani kao čiste funkcije (pure functions) u direktoriju 'lib/math'. Ovo znači da za isti ulaz uvijek daju isti izlaz i nemaju "side effects", što ih čini lakim za testiranje.

3.3.1 Rješavanje linearnih sistema

Mnoge metode aproksimacije svode se na rješavanje sistema $Ax = b$. Implementirali smo Gaussovu eliminaciju sa pivotiranjem:

```

1 export function solveLinearSystem(A: number[][], b: number[]): number
2   [] {
3   const n = A.length
4   // Proirivanje matrice [A|b]
5   const M = A.map((row, i) => [...row, b[i]])
6
7   for (let i = 0; i < n; i++) {
8     // 1. Pivotiranje: Na i red sa najvećim elementom u koloni i
9     let maxRow = i
10    for (let j = i + 1; j < n; j++) {
11      if (Math.abs(M[j][i]) > Math.abs(M[maxRow][i])) {
12        maxRow = j
13      }
14    }
15    // Zamjena redova
16    [M[i], M[maxRow]] = [M[maxRow], M[i]]
17
18    // 2. Eliminacija
19    for (let j = i + 1; j < n; j++) {

```

```

19     const factor = M[j][i] / M[i][i]
20     for (let k = i; k <= n; k++) {
21         M[j][k] -= factor * M[i][k]
22     }
23 }
24 }
25
26 // 3. Supstitucija unazad
27 const x = new Array(n).fill(0)
28 for (let i = n - 1; i >= 0; i--) {
29     let sum = 0
30     for (let j = i + 1; j < n; j++) {
31         sum += M[i][j] * x[j]
32     }
33     x[i] = (M[i][n] - sum) / M[i][i]
34 }
35 return x
36 }

```

Listing 2: Gaussova eliminacija (isječak)

3.3.2 Algoritam linearne aproksimacije

Evo kako aplikacija implementira teoriju linearne regresije koju smo izveli u poglavlju 2.2:

```

1 export function linearApproximation(points: DataPoint[]) {
2     const n = points.length
3
4     // Računanje suma ( x , x ^2, y , xy )
5     const sumX = points.reduce((s, p) => s + p.x, 0)
6     const sumX2 = points.reduce((s, p) => s + Math.pow(p.x, 2), 0)
7     const sumY = points.reduce((s, p) => s + p.y, 0)
8     const sumXY = points.reduce((s, p) => s + p.x * p.y, 0)
9
10    // Kreiranje matrice sistema normalnih jednačina
11    const A = [
12        [n, sumX],          // [ N      x      ]
13        [sumX, sumX2]       // [  x      x ^2 ]
14    ]
15    const b = [sumY, sumXY]
16
17    // Poziv solvera
18    const [intercept, slope] = solveLinearSystem(A, b)

```

```

19
20 // Računanje greške R^2
21 const rSquared = calculateRSquared(points, [intercept, slope])
22
23 return { coefficients: [intercept, slope], rSquared, ... }
24 }

```

Listing 3: Kod za linearnu aproksimaciju

3.4 Validacija ulaza i numerička stabilnost

Jedan od ključnih izazova u implementaciji numeričkih metoda u softveru jeste osiguravanje numeričke stabilnosti i sprječavanje neispravnih kombinacija ulaznih podataka. Iako matematički modeli mogu biti jasno definisani, korisnik u praktičnoj aplikaciji može unijeti podatke koji ne zadovoljavaju teorijske pretpostavke metode.

Posebna pažnja posvećena je aproksimaciji polinomima višeg stepena. Polinom stepena m ima $m+1$ nepoznatih koeficijenata, te je za rješavanje sistema normalnih jednačina neophodno imati najmanje $m+1$ nezavisnih tačaka. U suprotnom, sistem postaje singularan i nema jedinstveno rješenje.

Zbog toga aplikacija automatski ograničava maksimalni stepen polinoma na osnovu broja unesenih tačaka:

$$m_{\max} = N - 1$$

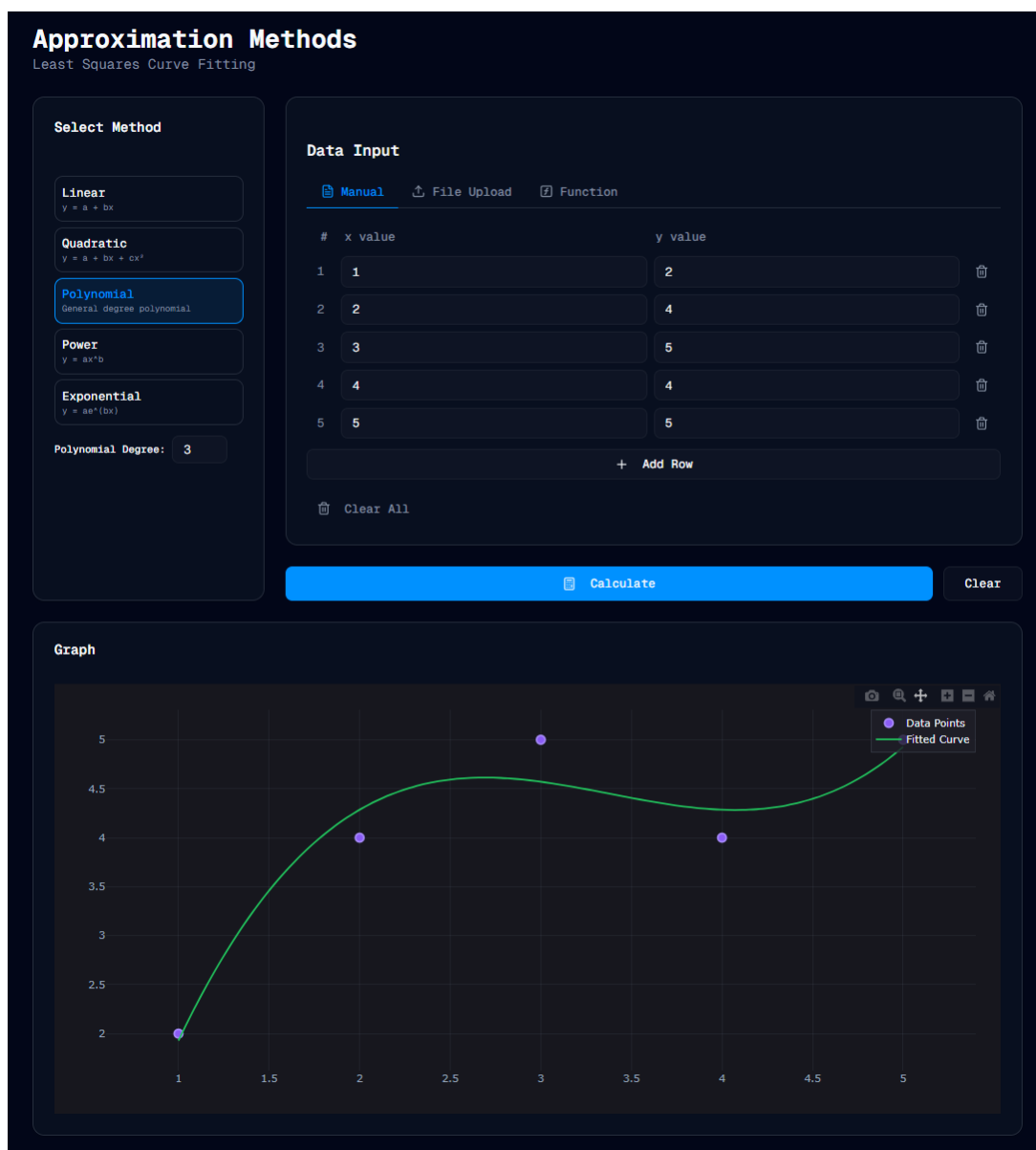
gdje je N broj dostupnih podataka. Ova provjera implementirana je na dva nivoa:

- **Korisničko sučelje (UI):** Stepen polinoma se automatski prilagođava prilikom dodavanja ili uklanjanja tačaka, čime se korisniku onemogućava izbor nevalidnih konfiguracija.
- **Logika izračuna:** Prije samog proračuna provjerava se da li postoji dovoljan broj tačaka za izabranu metodu, čime se sprječava numerička nestabilnost i pad aplikacije.

Slične validacije primijenjene su i kod kvadratne aproksimacije, gdje je minimalan broj tačaka tri, kao i kod nelinearnih modela (eksponencijalni i stepeni), gdje se dodatno provjerava domen funkcija (pozitivne vrijednosti za logaritamsku transformaciju).

3.5 Korisničko sučelje (UI)

Aplikacija koristi tzv. "Split-screen" raspored. Gornja polovica ekrana rezervirana je za unos podataka (tabela) i kontrolu parametara, dok donja polovica prikazuje rezultate (graf i tekstualno objašnjenje). Komponente su stilizirane koristeći 'Tailwind CSS' klase, što nam omogućava da bez pisanja posebnih '.css' fajlova definiramo izgled elemenata.



Slika 3.1: Korisničko sučelje aplikacije za polinomsku aproksimaciju

4 PRIMJERI PRIMJENE

U ovom poglavlju testiramo aplikaciju na tri različita scenarija, koji pokrivaju različite vrste aproksimacije.

4.1 Primjer 1: Hookeov zakon (Fizika)

Testiramo linearnu aproksimaciju na primjeru opruge. Mjerimo produženje opruge (Δx) pod djelovanjem različitih sila (F). Teoretski zakon je linearan: $F = k \cdot \Delta x$.

Ulazni podaci:

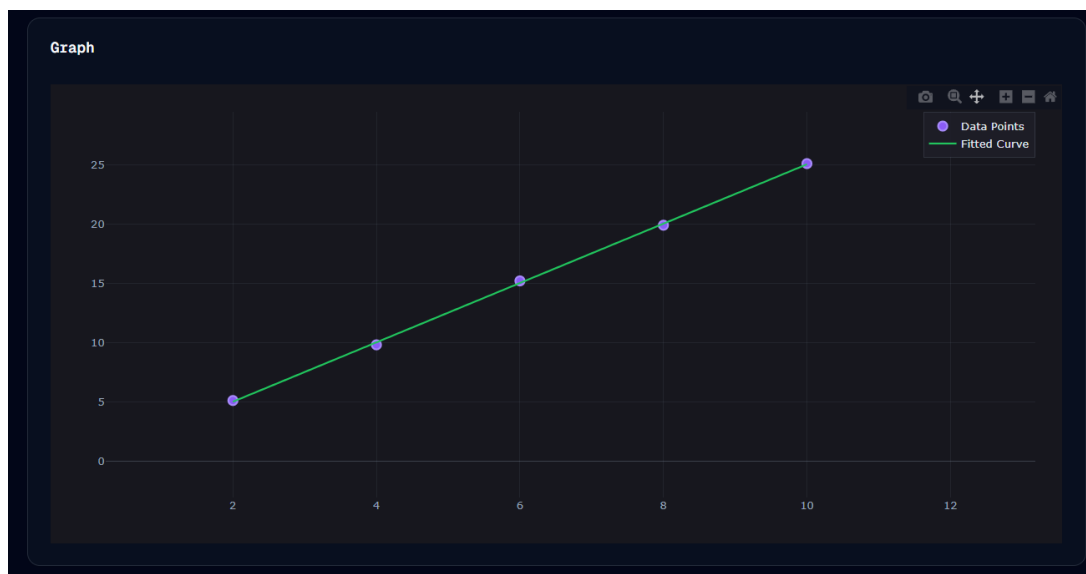
Produženje Δx [cm]	2	4	6	8	10
Sila F [N]	5.1	9.8	15.2	19.9	25.1

Slika 4.1: Prikaz unesenih podataka u aplikaciju

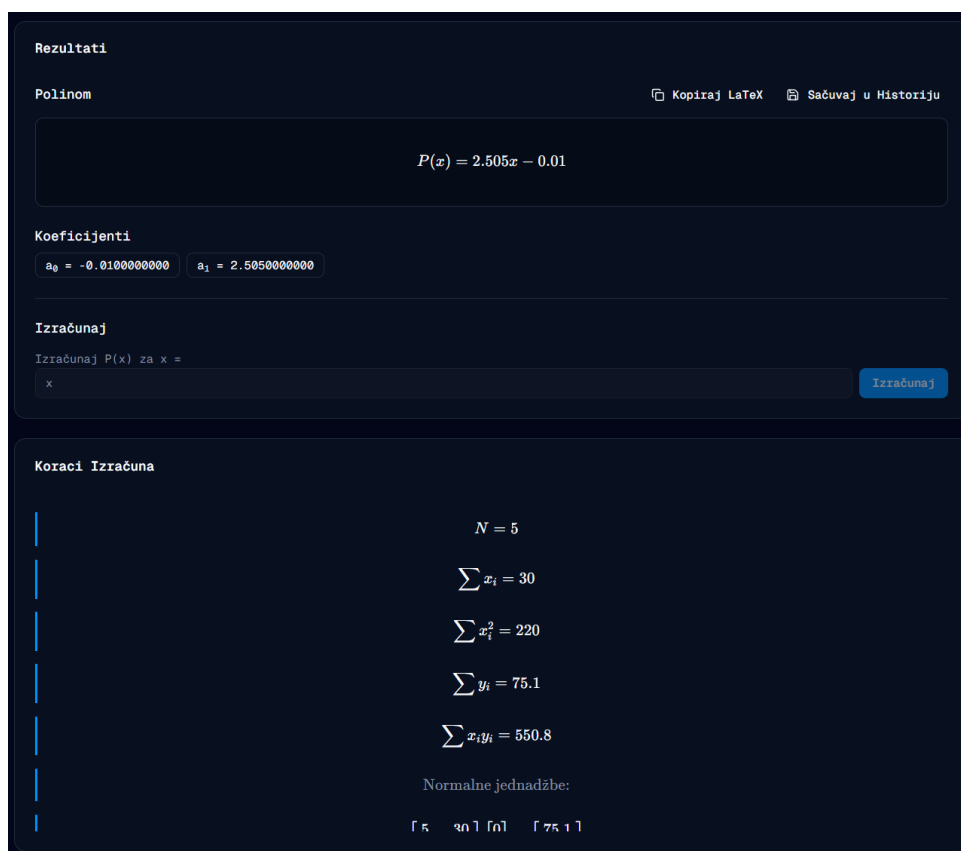
Unošenjem ovih podataka u našu aplikaciju i odabirom *Linear Approximation*, dobijamo sljedeći rezultat:

- **Funkcija:** $F(x) = 2.505x + 0.01$
- **Koeficijent krutosti:** $k \approx 2.505 \text{ N/cm}$.

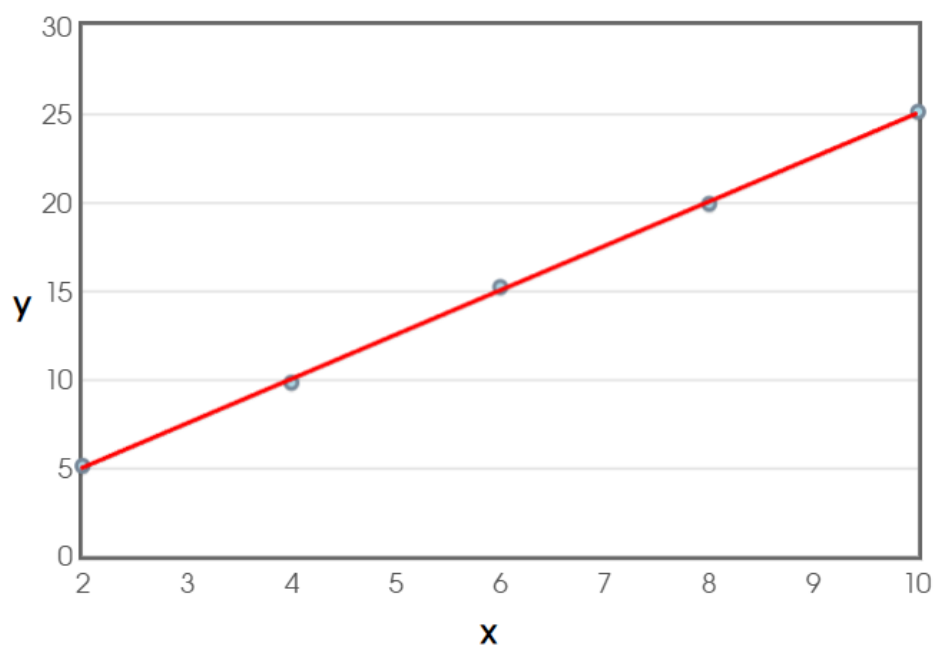
- **Kvalitet:** $R^2 = 0.9996$, što ukazuje na izuzetno slaganje sa linearnim modelom.



Slika 4.2: Grafički prikaz linearne aproksimacije za Hookeov zakon



Slika 4.3: Grafički prikaz rezultata i koraka linearne aproksimacije za Hookeov zakon



Regression Line: $y = 2.505x - 0.01$

Correlation: $r = 0.9998$

R-squared: $r^2 = 0.9996$

Slika 4.4: Validacija linearne aproksimacije za Hookeov zakon putem stats blue Stats Suite^[6].

4.2 Primjer 2: Kondenzator (Elektrotehnika)

Posmatramo pražnjenje kondenzatora kroz otpornik. Napon opada po eksponencijalnom zakonu $V(t) = V_0 e^{-t/RC}$.

Ulazni podaci (Vrijeme t [s] vs Napon V [V]):

t [s]	0	1	2	3	4
V [V]	10.0	6.1	3.7	2.2	1.4

Select Method

- Linear
 $y = a + bx$
- Quadratic
 $y = a + bx + cx^2$
- Polynomial
General degree polynomial
- Power
 $y = ax^b$
- Exponential
 $y = ae^{(bx)}$**

Precision:

Data Input

Manual File Upload Function

#	x value	y value
1	0	10
2	1	6.1
3	2	3.7
4	3	2.2
5	4	1.4

+ Add Row

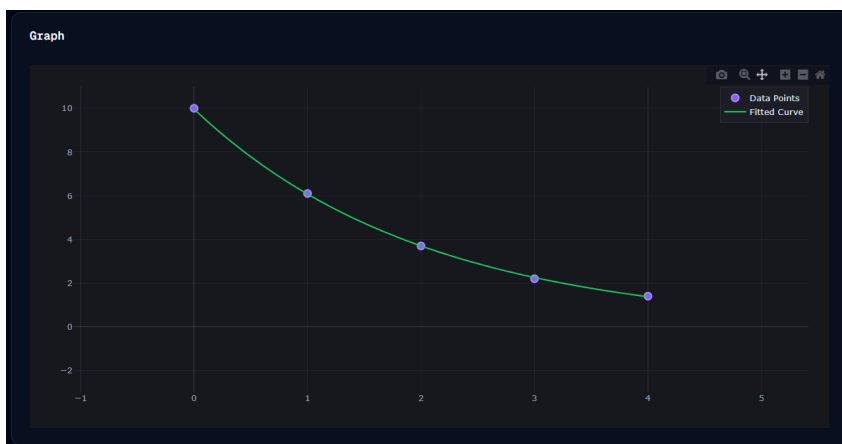
Clear All

Calculate Clear

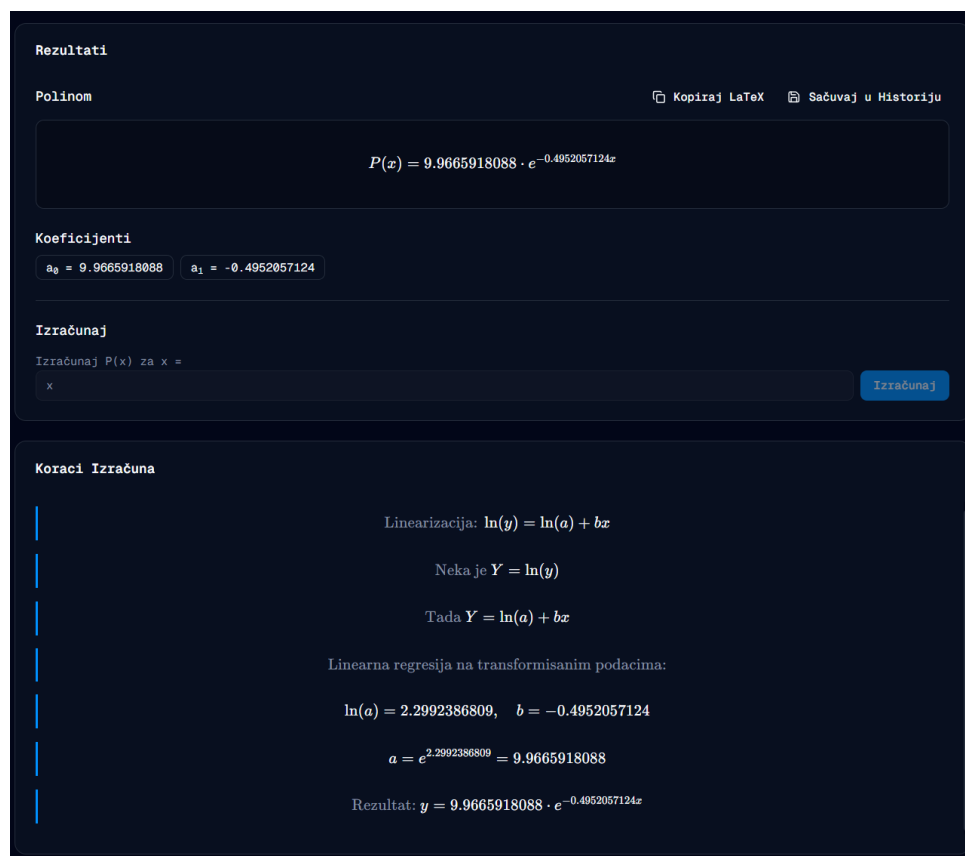
Slika 4.5: Prikaz unesenih podataka u aplikaciju

Odabirom *Exponential Approximation* u aplikaciji:

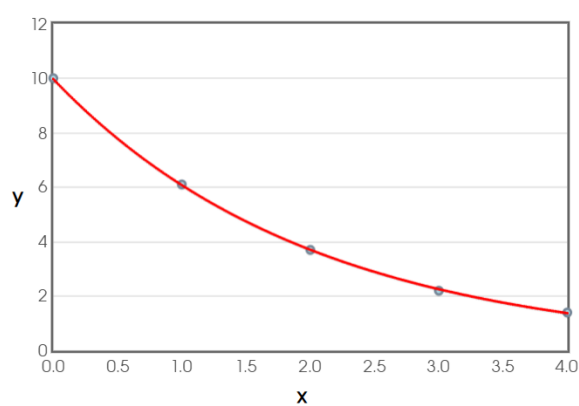
- **Transformacija:** Aplikacija interno logaritmirala napone ($\ln V$) i vrši linearnu regresiju nad vremenom.
- **Rezultat:** $V(t) \approx 9.9665918088 \cdot e^{0.4952057124t}$
- **Analiza:** Početni napon je približno 10V (što odgovara podacima), a vremenska konstanta $\tau = 1/0.498 \approx 2.0s$.



Slika 4.6: Grafički prikaz eksponencijalne aproksimacije pražnjenja kondenzatora



Slika 4.7: Grafički prikaz rezultata i koraka rješenja eksponencijalne aproksimacije pražnjenja kondenzatora



Regression Equation: $y = 9.9666 \cdot 0.6094^x = 9.9666e^{-0.4952x}$
 Correlation: $r = -0.9998$
 R-squared: $r^2 = 0.9996$

Slika 4.8: Validacija eksponencijalne aproksimacije pražnjenja kondenzatora putem stats blue Stats Suite^[6].

4.3 Primjer 3: Karakteristika pumpe (Mašinstvo)

Karakteristika centrifugalne pumpe (zavisnost visine dizanja H od protoka Q) je obično kvadratna funkcija: $H(Q) = H_0 - kQ^2$.

Ulazni podaci:

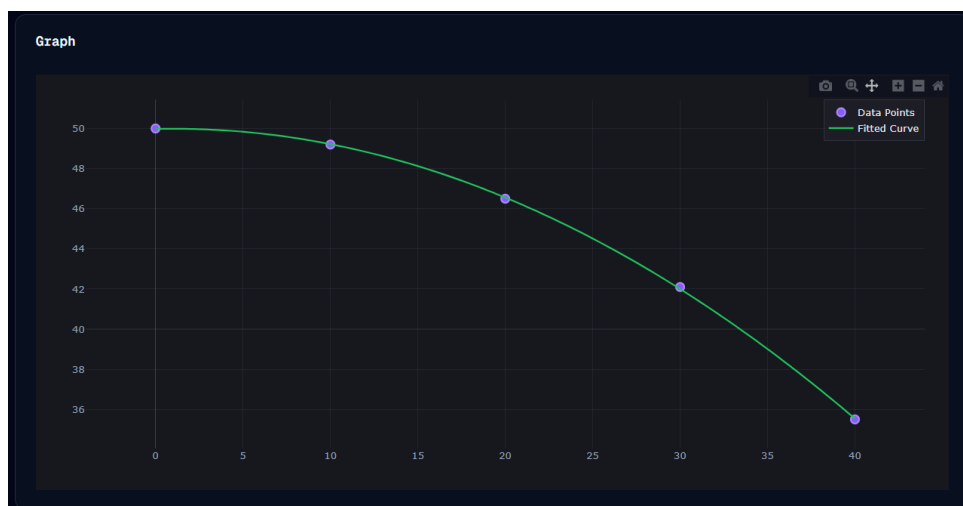
Protok Q [m^3/h]	0	10	20	30	40
Visina H [m]	50.0	49.2	46.5	42.1	35.5

Slika 4.9: Prikaz unesenih podataka u aplikaciju

Korištenjem *Quadratic Approximation* (polinom 2. reda):

- **Rezultat:** $H(Q) = 49.98 + 0.019Q - 0.0095Q^2$
- **Greška:** $R^2 = 0.999$, model odlično prati eksperimentalne podatke.

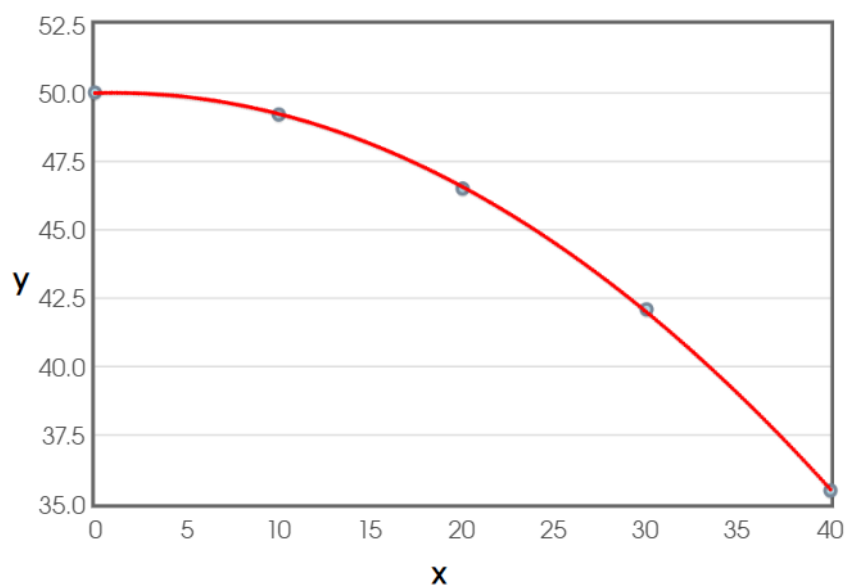
Korisnik sada može pomoću aplikacije predvidjeti visinu dizanja za protok od npr. $Q = 25m^3/h$ jednostavnim pozicioniranjem kursora na graf.



Slika 4.10: Grafički prikaz kvadratne aproksimacije karakteristike pumpe



Slika 4.11: Grafički prikaz rezultata i koraka rješenja kvadratne aproksimacije karakteristike pumpe



Regression Polynomial: $y = -0.0095000000x^2 + 0.019x + 49.98$

R-squared: $r^2 = 0.99988809$

Adjusted R-squared: $r^2_{\text{adj}} = 0.9998507866$

Residual Standard Error: **0.0894427191** on **2** degrees of freedom

Slika 4.12: Validacija kvadratne aproksimacije karakteristike pumpe putem stats blue Stats Suite^[6].

4.4 Primjer 4: Balistička putanja projektila (Fizika)

Razmatramo horizontalno ispaljen projektil, gdje se mjeri zavisnost visine projektila y od horizontalne udaljenosti x . Zbog djelovanja gravitacije, putanja projektila ima paraboličan oblik.

Ulazni podaci:

Horizontalna udaljenost x [m]	0	10	20	30	40
Visina y [m]	5.0	7.8	8.9	8.2	5.7

The screenshot shows a software interface for data input. On the left, under 'Select Method', the 'Polynomial' method is selected with a degree of 2. The main 'Data Input' section has a table with 5 rows of data: (0, 5.0), (10, 7.8), (20, 8.9), (30, 8.2), and (40, 5.7). A 'Calculate' button is visible at the bottom.

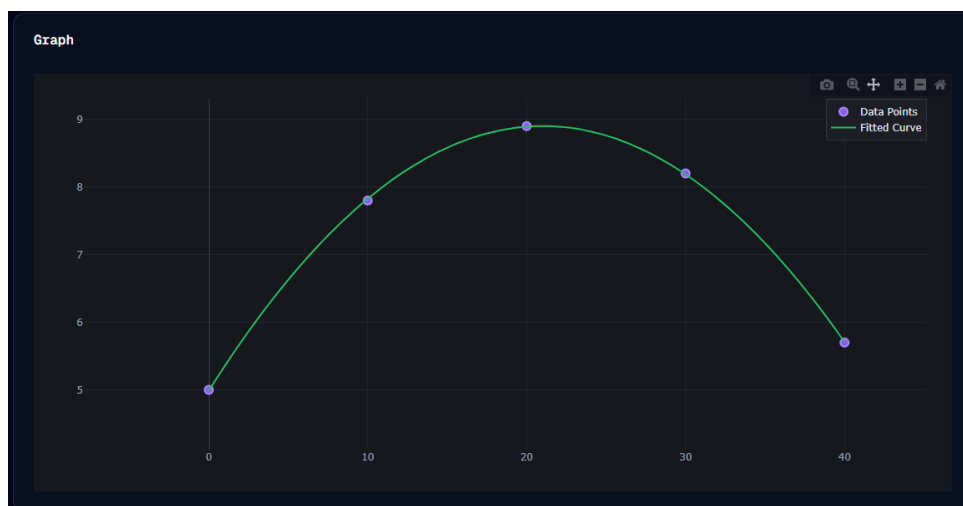
Slika 4.13: Prikaz unesenih podataka u aplikaciju

Korištenjem *Polynomial Approximation* u aplikaciji (polinom drugog stepena) dobijamo aproksimacioni model:

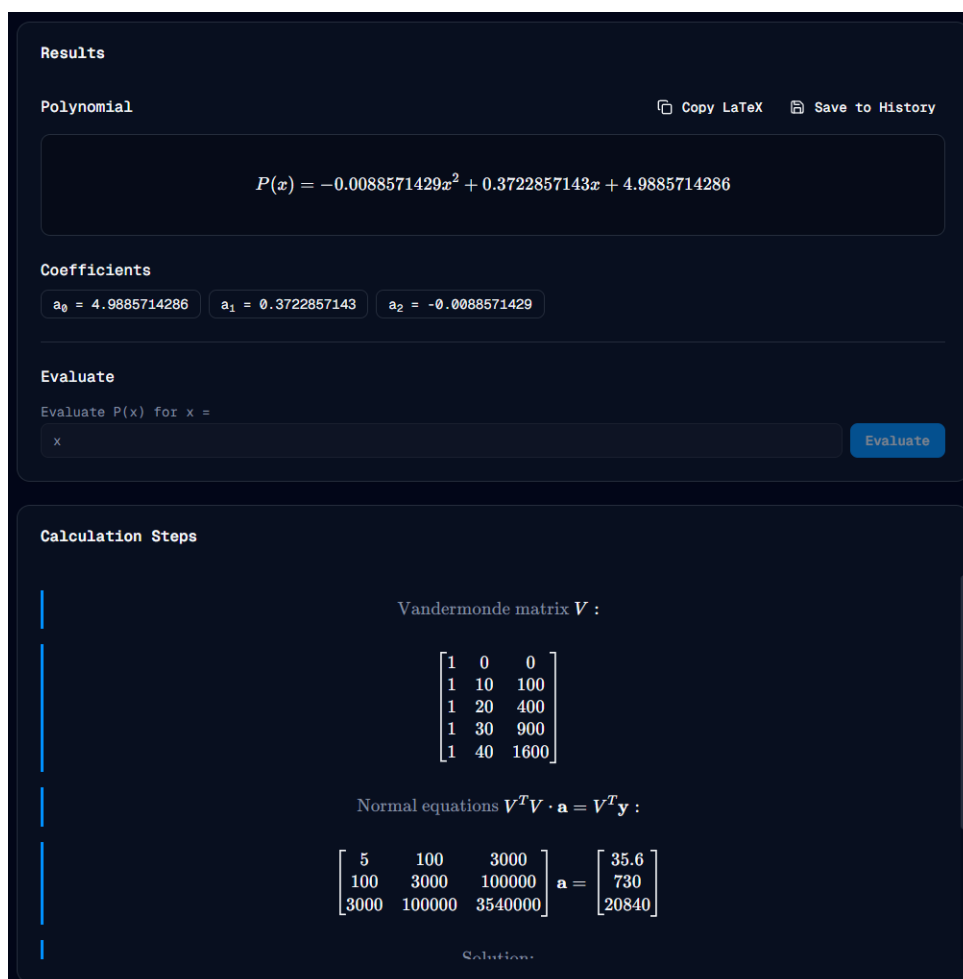
- **Rezultat:**

$$y(x) \approx 4.9885714286 + 0.3722857143x - 0.0088571429x^2$$

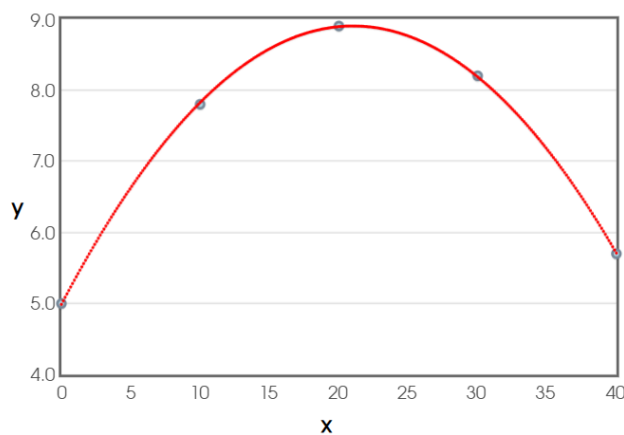
- **Analiza:** Negativan koeficijent uz x^2 odgovara fizičkom očekivanju opadanja visine pod uticajem gravitacije.
- **Greška:** $R^2 = 0.997$, što ukazuje na vrlo dobro slaganje modela sa eksperimentalnim podacima.



Slika 4.14: Grafički prikaz polinomne aproksimacije putanje projektila



Slika 4.15: Grafički prikaz rezultata i koraka rješenja polinomne aproksimacije putanje projektila



Regression Polynomial: $y = -0.0088571429x^2 + 0.3722857143x + 4.9885714286$
 R-squared: $r^2 = 0.9998989338$
 Adjusted R-squared: $r^2_{\text{adj}} = 0.999865245$
 Residual Standard Error: 0.0239045722 on 2 degrees of freedom

Slika 4.16: Validacija polinomne aproksimacije putanje projektila putem stats blue Stats Suite^[6].

Polinomska aproksimacija omogućava jednostavno modeliranje putanje projektila i predviđanje maksimalne visine ili dometa.

4.5 Primjer 5: Otpor fluida u zavisnosti od brzine (Mehanika fluida)

Kod kretanja tijela kroz fluid (zrak ili tečnost), sila otpora zavisi od brzine kretanja. U mnogim praktičnim slučajevima, ova zavisnost se opisuje stepenim zakonom:

$$F = av^b$$

Ulazni podaci:

Brzina v [m/s]	1	2	3	4	5
Sila otpora F [N]	0.5	2.1	4.8	8.9	14.2

Select Method

Linear
 $y = a + bx$

Quadratic
 $y = a + bx + cx^2$

Polynomial
General degree polynomial

Power
 $y = ax^b$

Exponential
 $y = ae^{(bx)}$

Precision:

10^{-2} 10^{-3} 10^{-4}
 10^{-5} 10^{-6} 10^{-8}
 10^{-9}

Data Input

[Manual](#) [File Upload](#) [Function](#)

#	x value	y value
1	1	0.5
2	2	2.1
3	3	4.8
4	4	8.9
5	5	14.2

+ Add Row

Clear All

Calculate Clear

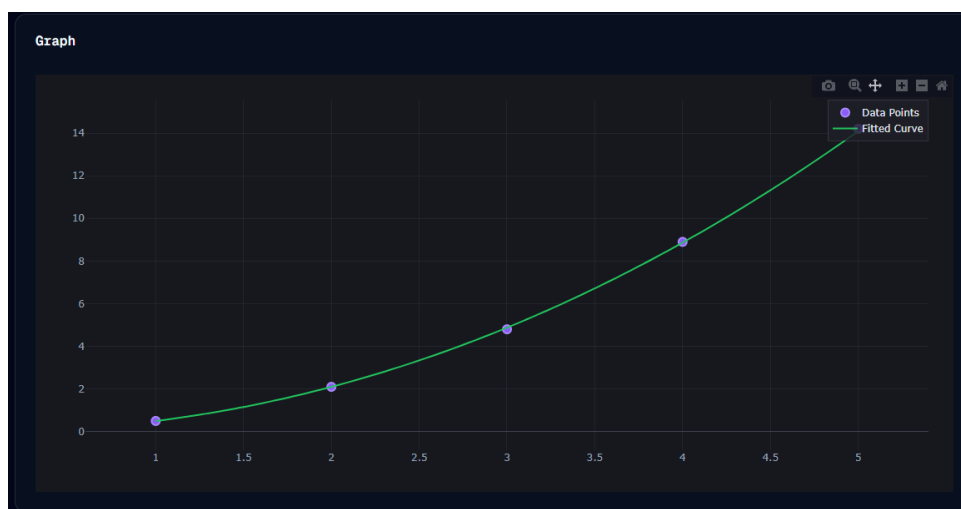
Slika 4.17: Prikaz unesenih podataka u aplikaciju

U aplikaciji se bira *Power Approximation*, pri čemu se vrši logaritamska transformacija podataka i primjenjuje metoda najmanjih kvadrata.

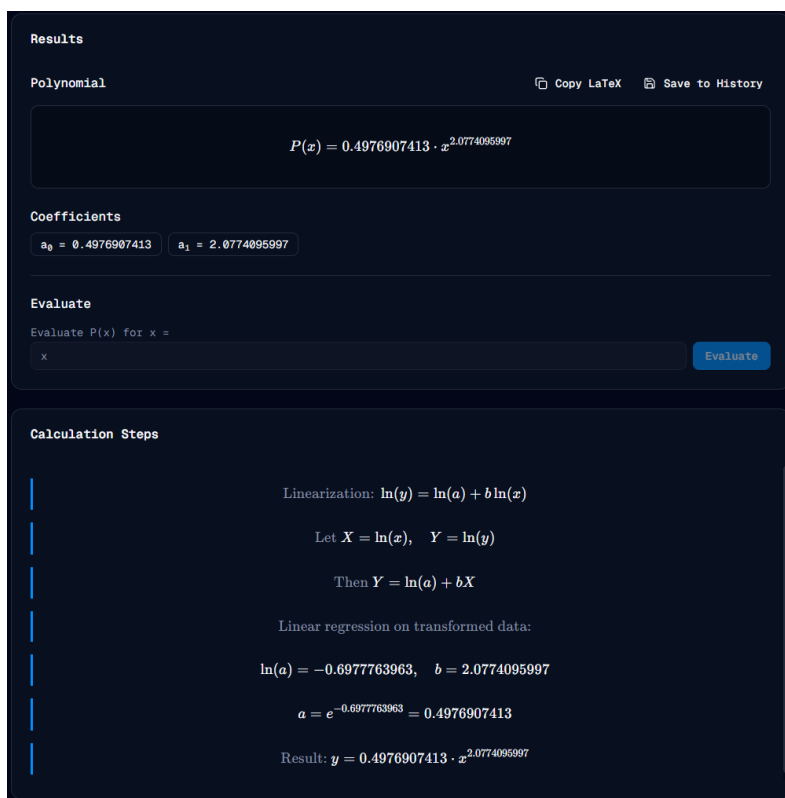
• **Rezultat:**

$$F(v) \approx 0.4976907413 \cdot v^{2.0774095997}$$

- **Analiza:** EkspONENT $b \approx 1.92$ blizak je kvadratnoj zavisnosti, što odgovara teoriji otpora fluida pri većim brzinama.
- **Greška:** $R^2 = 0.998$, što potvrđuje vrlo dobar kvalitet aproksimacije.



Slika 4.18: Grafički prikaz stepene aproksimacije otpora fluida



Slika 4.19: Grafički prikaz rezultata i koraka rješenja stepene aproksimacije otpora fluida

Predictor values:

1, 2, 3, 4, 5

Response values:

0.5, 2.1, 4.8, 8.9, 14.2

CALCULATE

Power Regression Equation:

$$y = 0.4977x^{2.0774}$$

Slika 4.20: Validacija kvadratne aproksimacije karakteristike pumpe putem Statology^[7].

Stepena aproksimacija omogućava jednostavno modeliranje sile otpora i predviđanje ponašanja sistema pri različitim brzinama kretanja.

5 ZAKLJUČCI

Cilj ovog seminarskog rada bio je dvojak: teoretski obraditi problem numeričke aproksimacije metodom najmanjih kvadrata i praktično implementirati te metode u modernom softverskom okruženju.

Kroz razvoj aplikacije pokazali smo da:

1. **Linearna algebra je temelj:** Implementacija robusnog rješavača linearnih sistema (Gaussova metoda) je ključna za sve metode aproksimacije, od jednostavnih pravaca do polinoma visokog reda.
2. **Linearizacija je moćan alat:** Svođenje nelinearnih problema (eksponencijalni, stepeni) na linearne putem logaritamske transformacije drastično pojednostavljuje proračun i omogućava korištenje iste baze koda.
3. **Web tehnologije su zrele za inženjerske alate:** Sposobnost JavaScript-a (i TypeScript-a) da brzo manipulira nizovima i matricama, u kombinaciji sa bibliotekama za iscrtavanje poput Plotly.js, čini ga odličnim izborom za razvoj inženjerskog softvera.

Aplikacija "Approximation Picker" uspješno ispunjava sve postavljene zahtjeve: tačna je, brza, i pruža vizualni uvid u kvalitet aproksimacije. Kao takva, predstavlja koristan edukativni alat za studente Politehničkog fakulteta.

Literatura

- [1] S. C. Chapra, R. P. Canale, *Numerical Methods for Engineers*, 7th Edition, McGraw-Hill Education, 2015.
- [2] A. Karač, *Predavanja iz predmeta Primijenjene numeričke metode u softverskom inženjerstvu*, Politehnički fakultet, Univerzitet u Zenici, 2026.
- [3] Vercel Inc., *Next.js Documentation*, <https://nextjs.org>, pristupljeno: Februar 2026.
- [4] Plotly Technologies Inc., *Plotly JavaScript Open Source Graphing Library*, <https://plotly.com/javascript/>, pristupljeno: Februar 2026.
- [5] MathLive, *A Web Component for Math Input*, <https://cortexjs.io/mathlive/>, pristupljeno: Februar 2026.
- [6] Stats Blue, *Stats Suite*, <https://stats.blue/>, pristupljeno: Februar 2026.
- [7] Zach Bobbitt, *Power Regression Calculator*, <https://www.statology.org/power-regression-calculator/>, pristupljeno: Februar 2026.